MAGNIT TECH

Лекция 4
SQL для дата-инженера

часть 2
планы, индексы, звезды

Курс Data Engineering | MAGNIT TECH x НИУ ВШЭ

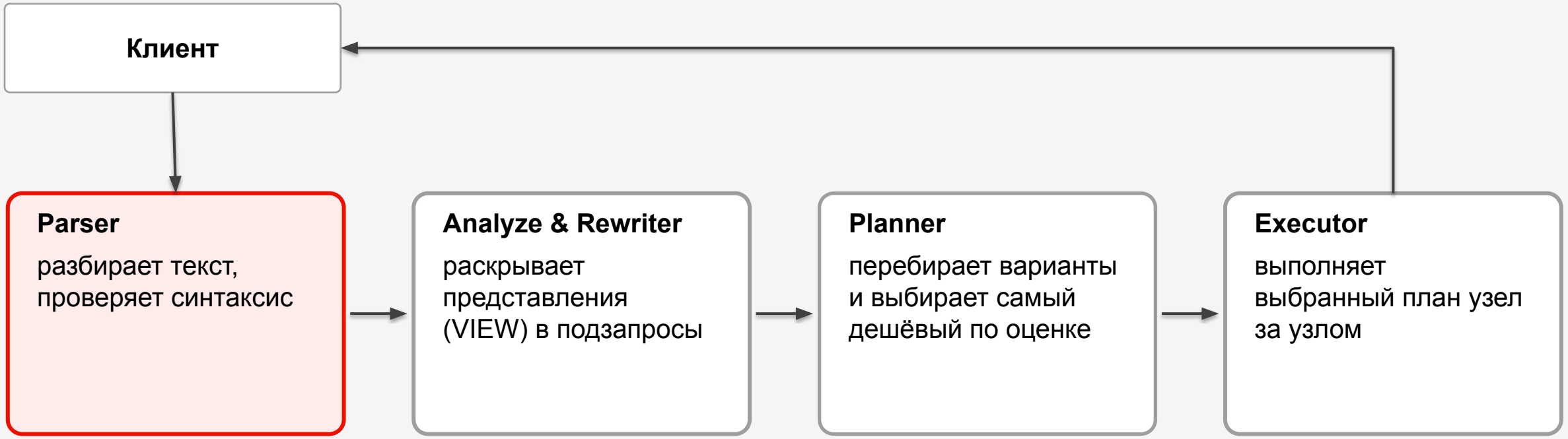
План лекции

- Как база выполняет запрос?
- Как читать EXPLAIN?
- Почему план бывает плохим?
- Что такое индекс и когда он помогает?
- Как ускорить запрос: типовые ошибки.
- Что такое факты и измерения?
- Чем «снежинка» отличается от «звезды»?
- Что делать, если данные в измерении меняются?

MAGNIT TECH

Как PostgreSQL выполняет запрос

Путь запроса внутри PostgreSQL



SQL декларативен: мы описываем, что получить. Как это сделать — какой индекс взять, в каком порядке и каким алгоритмом соединять таблицы — решает планировщик.

Parser

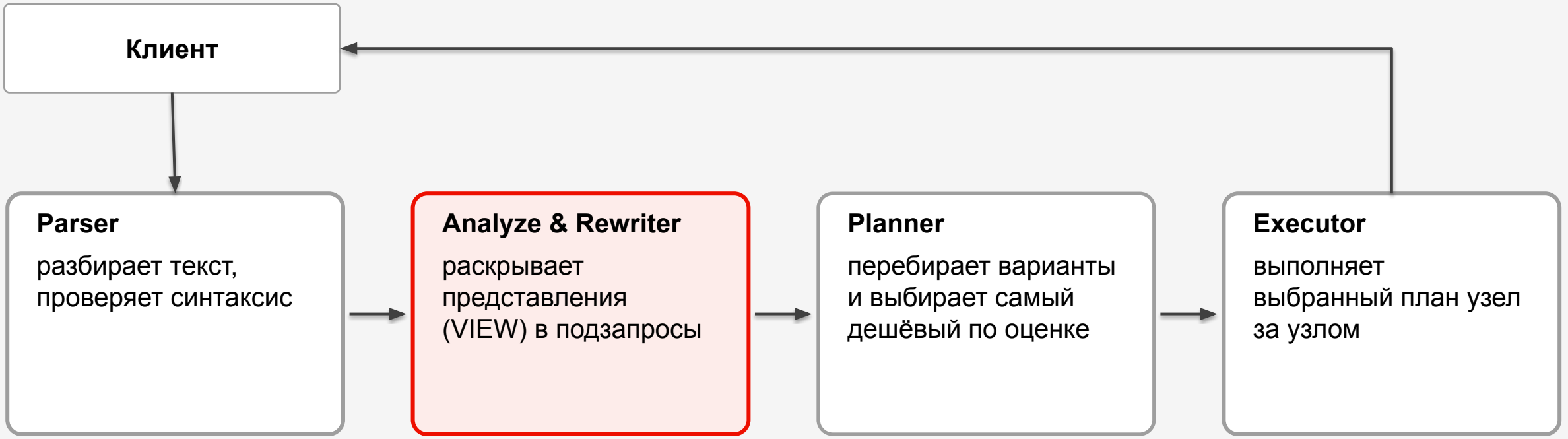
Проверяет синтаксис и отвечает на вопрос “Это вообще SQL?”
Существует ли таблица orders парсеру не важно

```
SELEC order_id, status
FROM orders
```

```
SQL Error [42601]: ERROR: syntax error at or near
"SELEC"
  Position: 1

Error position: line: 1
```

Путь запроса внутри PostgreSQL



Analyze & Rewriter

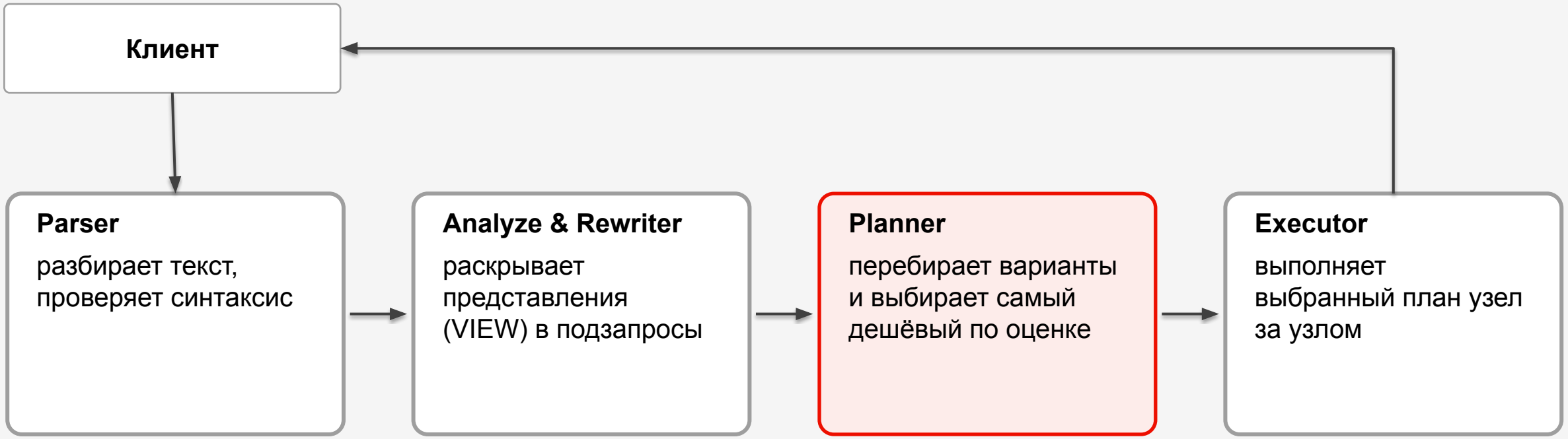
Сверяется с каталогом данных и раскрывает прелставления. Отвечает на вопросы “Таблица и столбцы существуют? Типы сходятся? Нет ли здесь VIEW?”

VIEW (представление) - сохраненный под именем запрос, к которому можно обращаться по имени как к таблице. Данные в представлении не хранятся.

```
create view orders_status as
select order_id, status
from orders

select * from orders_status
where status = 'paid'
```

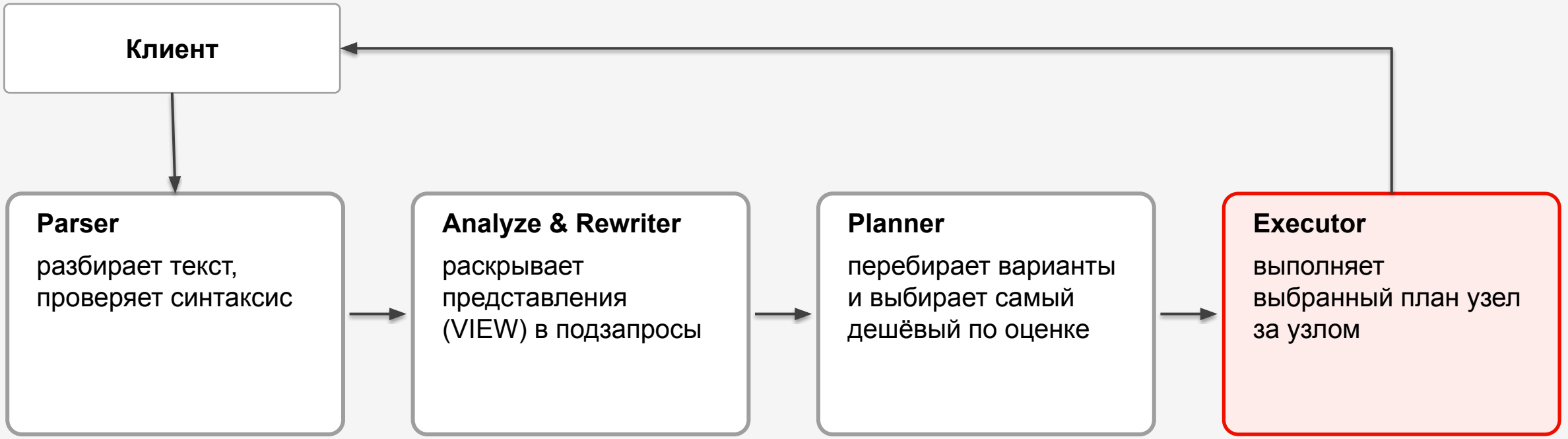
Путь запроса внутри PostgreSQL



Planner

Строит план выполнения запроса, опираясь на информацию из каталога об объектах, которые участвуют в запросе, и использует статистику.
План описывает как должен выполняться запрос исходя из стоимости каждого шага, выбирая самый дешевый план.

Путь запроса внутри PostgreSQL



Executor

Выполняет план по шагам: читает индексы и данные, сортирует, считает суммы — и отдаёт результат клиенту. Здесь возникают ошибки, которые зависят **от самих данных**.

```sql
select o.order_id,
       o.total_amount / (select count(*)
         from payments p
         where p.order_id = o.order_id
           and p.payment_status = 'success')
from orders o
where o.status = 'paid';
-- ERROR: division by zero
```

Синтаксис верный, план построен. Ошибка появится, только когда дойдём до заказа 119: он paid, но без успешной оплаты.

```sql
delete from orders
where status = 'test';
-- ERROR: update or delete on table
--   "orders" violates foreign key
--   constraint "order_items_order_id_fkey"
-- DETAIL: Key (order_id)=(115) is
--   still referenced from "order_items".
```

Нарушение внешнего ключа — тоже ошибка исполнителя.

Стадии запроса по ошибкам

Удобно запомнить стадии по тому, какие ошибки на них возникают — все примеры из базы shop.

Стадия	Вопрос	Типичная ошибка
Parser	Это вообще SQL?	`syntax error at or near "selec"`
Analyze	Таблица и столбцы существуют?	`relation "order_item" does not exist` `operator does not exist: text = integer`
Rewriter	Нет ли тут VIEW?	—
Planner	Как выполнить быстрее?	ошибки нет, но план может быть медленным
Executor	Выполняю план	`division by zero`, нарушение внешнего ключа



Результат идёт клиенту сразу от исполнителя; главное решение принимает планировщик.

MAGNIT TECH

EXPLAIN:
читаем план выполнения



Три режима EXPLAIN

EXPLAIN

Останавливается после планировщика: показывает план и прогноз, **ничего не выполняет**.

```
explain
select * from orders
where status = 'paid';
```

```
Seq Scan on orders
  (cost=0.00..3.76 rows=107)
  Filter: (status = 'paid')
```

EXPLAIN ANALYZE

Реально выполняет запрос и показывает факт рядом с прогнозом. Изменения применятся!

```
begin;
explain analyze
update orders
set status = 'cancelled'
where status = 'test';
rollback;
```

EXPLAIN (ANALYZE, BUFFERS)

Плюс страницы по 8 КБ: `hit` — из памяти PostgreSQL, `read` — с диска или из кэша ОС. Вариант по умолчанию.

Через `explain analyze delete` получим ту самую ошибку внешнего ключа: EXPLAIN ANALYZE действительно доходит до исполнителя.

Учебная база слишком маленькая — увеличим её

В `orders` всего **141 строка**, 2 страницы по 8 КБ. Любой запрос идёт за доли миллисекунды через Seq Scan.

Размножим заказы до 5 млн строк в `orders_big`: каждый заказ копируем 36 000 раз, клиенты получают номера до 40 000, даты немного сдвигаются.

Автосбор статистики выключен — он пригодится в третьем вопросе.

```
create table orders_big
  with (autovacuum_enabled = false) as
select (g - 1) * 1000 + o.order_id  as order_id,
       o.customer_id + 40 * (g % 1000) as customer_id,
       o.order_created_at
         - ((g * 7919) % 60) * interval '1 day'
         + (g % 1440) * interval '1 minute'
                                       as order_created_at,
       o.status, o.total_amount
from orders o
cross join generate_series(1, 36000) as g;

alter table orders_big add primary key (order_id);
analyze orders_big;
```

5 076 000

строк в orders_big

292 МБ

размер таблицы

Скрипт со всеми примерами: lecture4_demo.sql. Время у вас будет другим, структура планов — та же.

Разбираем план: последние заказы клиента

```
set max_parallel_workers_per_gather = 0;

explain (analyze, buffers)
select order_id, order_created_at, total_amount
from orders_big
where customer_id = 4217
order by order_created_at  desc
limit 20;
```

План — дерево шагов, читаем его **снизу вверх**: Seq Scan читает всю таблицу → Sort сортирует найденное → Limit берёт 20 строк.

Параллельность пока выключили — так план проще. Включим её на следующем слайде.

```
Limit (cost=100822.91..100822.96 rows=20) (actual time=143.347..143.349 rows=20 loops=1)
  Buffers: shared hit=3 read=37376
  -> Sort (cost=100822.91..100823.39 rows=193) (actual time=124.181..124.182 rows=20 loops=1)
        Sort Key: order_created_at DESC
        Sort Method: top-N heapsort  Memory: 27kB
        -> Seq Scan on orders_big (cost=0.00..100817.77  rows=193)
                                  (actual time=0.461..124.133   rows=108 loops=1)
              Filter: (customer_id = 4217)
              Rows Removed by Filter: 5075892
              Buffers: shared  read=37376
Execution Time: 163.157 ms
```

MAGNIT TECH

Что означает каждая часть строки

Что в плане	Что это значит
`cost=0.00..100817.77`	оценка «стоимости» в условных единицах, нужна для сравнения вариантов. **Не миллисекунды!**
`rows=193` в первой скобке	сколько строк ожидал планировщик
`rows=108` во второй скобке	сколько строк получилось на самом деле
`actual time=0.461..124.133`	мс до первой и до последней строки узла
`loops=1`	сколько раз выполнялся узел; если больше 1 — умножаем строки и время на loops
`Rows Removed by Filter`	сколько строк прочитали и выбросили: 5 075 892 — почти вся таблица
`Buffers: hit=3 read=37376`	страницы из памяти и с диска
`Execution Time: 163.157 ms`	общее время выполнения запроса

Тот же план с параллельностью

```
reset max_parallel_workers_per_gather;

-- тот же запрос: последние 20 заказов
-- клиента 4217 из orders_big
```

Таблицу читают три процесса: основной и 2 помощника. Каждый читает свою треть, Gather Merge собирает результаты вместе.

loops=3 — цифры в шаге на один процесс: 36 × 3 = 108 строк, 1 691 964 × 3 = 5 075 892.

50 мс вместо 163, но таблица всё равно прочитана целиком.

```
Limit (cost=64812.23..64814.56 rows=20) (actual time=48.627..50.510 rows=20 loops=1)
  Buffers: shared hit=145 read=37343
  -> Gather Merge (cost=64812.23..64830.89 rows=160) (actual time=48.626..50.508 rows=20 loops=1)
        Workers Launched: 2
        -> Sort (cost=63812.20..63812.40 rows=80) (actual time=43.006..43.007 rows=17 loops=3)
              Sort Key: order_created_at DESC
              Sort Method: quicksort  Memory: 27kB
              -> Parallel Seq Scan on orders_big (cost=0.00..63810.07 rows=80)
                                        (actual time=2.128..42.935  rows=36 loops=3)
                    Filter: (customer_id = 4217)
                    Rows Removed by Filter: 1691964
                    Buffers: shared hit=33  read=37343
Execution Time: 50.526 ms
```

MAGNIT TECH

Что искать в любом плане

1 **Самый долгий шаг**
ищем, где actual time резко растёт; не забываем умножать на loops

2 **Прогноз и факт**
rows в двух скобках отличаются в десятки раз — планировщик ошибся

3 **Лишнее чтение**
большое Rows Removed by Filter у Seq Scan по большой таблице — скорее всего, не хватает индекса

4 **Сброс на диск**
external merge Disk — сортировке не хватило памяти, и она ушла на диск. Это медленно

Параллельный план: узлы `Gather` / `Gather Merge` и `loops > 1`. Цифры в узле — на один процесс: умножаем строки и время на loops.

108 из 5 млн
строк нашли в примере, прочитав всю таблицу

37 376 страниц
≈ 292 МБ с диска — вся таблица

193 vs 108
оценка близка: статистика в порядке

MAGNIT TECH

Почему план бывает плохим?

Устаревшая статистика

Ночью в `orders_big` догрузили **1 001 100** августовских заказов и сразу запустили следующий шаг.

ANALYZE не отработал (у нас он выключен) — планировщик думает, что заказов позже июля почти нет.

```
explain analyze
select count(*) from orders_big
where order_created_at >= '2026-08-01';
```

```
Parallel Seq Scan on orders_big
  (cost=0.00..76381.22  rows=253)
  (actual time=114.824..135.354  rows=333700 loops=3)
```

253 → 333 700

ожидали и получили на процесс: ошибка больше чем в 1300 раз

Все следующие шаги плана построены для маленькой выборки. Навигатор не глупый — у него старые данные.

MAGNIT TECH

До и после ANALYZE

```sql
select c.city, count(*)
from orders_big ob
join customers c on c.customer_id = ob.customer_id % 40 + 1
where ob.order_created_at >= '2026-08-01'
group by c.city;
```

	До ANALYZE	После ANALYZE
Ожидание строк после фильтра	253 на процесс	406 578 на процесс
Группировка	Sort + GroupAggregate	HashAggregate
Сортировка	333 700 строк, external merge Disk: 4248kB	6 строк, quicksort Memory: 25kB
Execution Time	170 мс	122 мс

```sql
insert into orders_big select ... ;
analyze orders_big;
```

Загрузили — обновили статистику. На миллиарде строк ошибка того же масштаба стоит часов.

MAGNIT TECH

Индексы:
самое главное



Индекс — это предметный указатель

Чтобы найти слово, мы не листаем всю книгу: открываем указатель, смотрим номер страницы и идём туда.

Индекс — отдельная структура: значения столбца лежат **по порядку**, рядом с каждым — адрес строки в таблице.

Чем платим:

- местом на диске;
- каждый INSERT, UPDATE и DELETE обновляет все индексы таблицы.

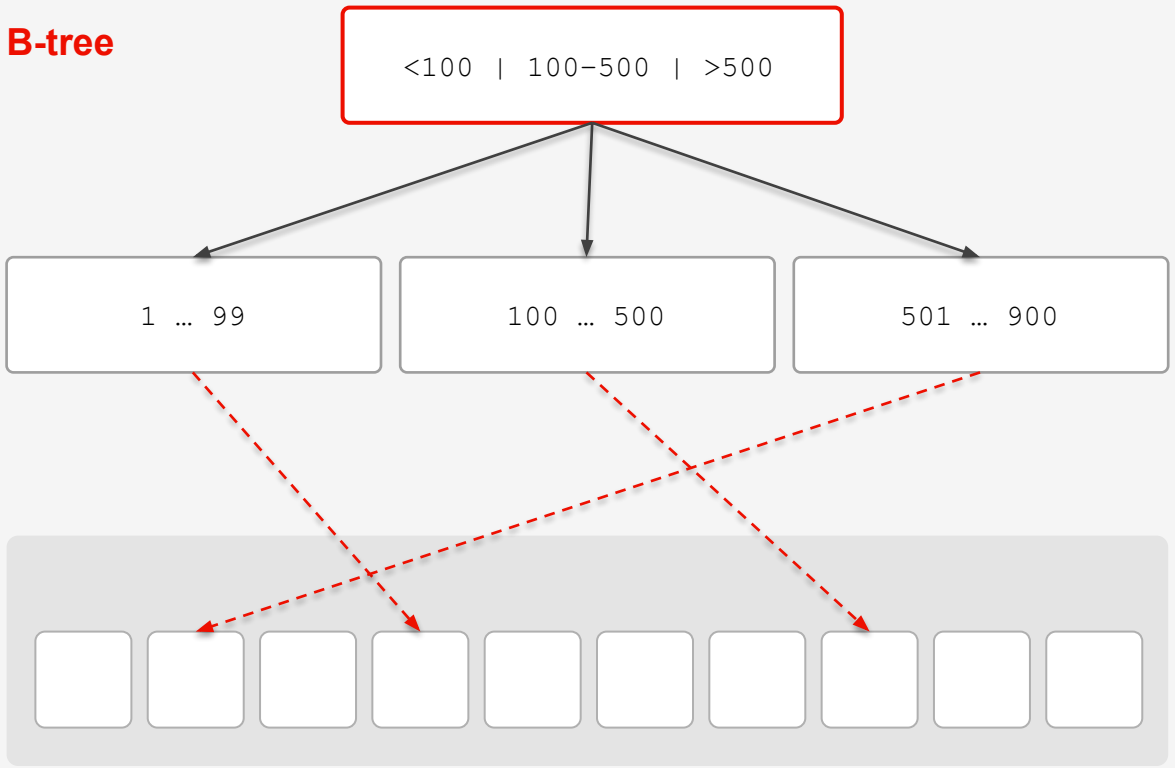


таблица: строки лежат в порядке вставки

Возвращаемся к примеру: создаём индекс

```
create index idx_orders_big_customer_created
    on orders_big (customer_id, order_created_at desc);
```

```
Limit (cost=0.43..81.17 rows=20) (actual time=0.025..0.095 rows=20 loops=1)
  Buffers: shared hit=3 read=20
  -> Index Scan using idx_orders_big_customer_created on orders_big
         (cost=0.43..779.51 rows=193) (actual time=0.024..0.093 rows=20 loops=1)
       Index Cond: (customer_id = 4217)
Execution Time: 0.105 ms
```

163 → 0,1 мс

время запроса

37 379 → 23

страниц прочитано

Было 163 мс (50 мс с параллельностью). Заказы клиента 4217 в индексе уже отсортированы по дате, поэтому из плана исчезли сортировка и параллельные процессы.

Индекс помогает, когда строк мало

```
create index idx_orders_big_status on orders_big (status);
```

Условие	Строк	Доля	План
`customer_id = 4217`	108	0,002 %	Index Scan — индекс очень помогает
`status = 'test'`	216 000	4,3 %	индекс (вариант Bitmap Scan)
`status = 'paid'`	3 852 000	76 %	Seq Scan — индекс есть, но база его не берёт

Чтобы достать строку через индекс, нужно два действия: найти запись в индексе и перейти по адресу в таблицу. 108 переходов — быстро. 3,85 млн переходов — проще прочитать таблицу подряд.

Для 4 % строк база тоже берёт индекс, но в варианте **Bitmap Scan**: сначала отмечает нужные страницы, потом читает их по порядку.

На маленькой таблице индекс не нужен

У исходной `orders` есть первичный ключ, но поиск по нему всё равно идёт через Seq Scan.

Таблица занимает 2 страницы: прочитать их подряд дешевле, чем открыть сначала индекс, а потом таблицу.

```
explain select * from orders where order_id = 113;
```

```
Seq Scan on orders (cost=0.00..3.76 rows=1 width=26)
  Filter: (order_id = 113)
```



Планировщик иногда «игнорирует» индекс — и чаще всего он прав.

Составной индекс: правило левого префикса

Индекс `(customer_id, order_created_at)` устроен как телефонный справочник: сначала по фамилии (клиенту), внутри — по имени (дате). Найти всех Петров без фамилии не получится.

Условие	Индекс поможет?	Что в плане
`where customer_id = 4217`	да	Index Scan, 0,1 мс
`where customer_id = 4217 and order_created_at >= '2026-06-01'`	да	Bitmap Index Scan, 0,05 мс
`where order_created_at >= '2026-07-01'`	почти нет: нет первого столбца	Parallel Seq Scan, 57 мс



*Первым ставим столбец, который сравниваем **на равенство**, за ним — столбец для **диапазона или сортировки**.*

Какие бывают индексы

B-tree

По умолчанию, подходит почти всегда: равенство, больше-меньше, сортировка, `like 'abc%'`. Все индексы базы shop — B-tree.

GIN

Поиск «внутри» значения: ключи в jsonb, элементы массива, слова в тексте.

BRIN

Огромные таблицы, куда данные пишутся по времени. Хороший кандидат — `raw_events` по `loaded_at`, будь там миллиарды строк.

Есть и другие (Hash, GiST, SP-GiST) — для специальных задач вроде геоданных.

Когда индекс не нужен

- **таблица маленькая** — как все таблицы базы shop;
- **запрос выбирает большую часть строк** — как `status = 'paid'`;
- **в таблицу много пишут**, а по столбцу почти не ищут.

Правило для начала

Создаём индекс под конкретный медленный запрос, а не «на всякий случай». Всегда сравниваем план до и после.

Индекс не бесплатен: место на диске и обновление при каждом INSERT, UPDATE и DELETE.

MAGNIT TECH

Как ускорить запрос:
типовые ошибки

Не прячем столбец в функцию

```
create index idx_orders_big_created on orders_big (order_created_at);
```

Плохо: индекс не работает

```
where date(order_created_at)
      = '2026-06-15'
```

73 мс

Parallel Seq Scan, Rows Removed by Filter: 1 686 600 × 3

Хорошо: столбец стоит «голым»

```
where order_created_at >= '2026-06-15'
  and order_created_at <  '2026-06-16'
```

26 мс

Bitmap Index Scan on idx_orders_big_created

То же с арифметикой: `total_amount > 1000 / 1.2`, *а не* `total_amount * 1.2 > 1000`. *И* `lower(status)` *ломает и индекс, и оценку строк.*

Не умножаем строки соединением

Ошибка из нашей практики: у заказа с двумя платежами строка задваивается.

```sql
select sum(o.total_amount)
from orders o
left join payments p using (order_id)
where o.order_id in (113, 116, 119);
```

5200 ≠ 3100

на самом деле 200 + 1900 + 1000 = 3100

Запрос выдаёт неверный ответ и при этом обрабатывает лишние строки. **Сначала группируем payments до одной строки на заказ, потом соединяем.**

Не берём лишнее

`select *` тянет все столбцы. В колоночных базах каждый лишний столбец — лишнее чтение с диска.

Сравниваем с тем же типом

`session_id` — текст: `where session_id = 's-1'`, а не `= 1`. Иначе ошибка или преобразование типов без индекса.

Как работать с медленным запросом

1

Найти

расширение pg_stat_statements собирает статистику по всем запросам

2

Снять план

explain (analyze, buffers)

3

Найти узел

где уходит время; сравнить оценку строк с фактом

4

Исправить

статистика → текст запроса → индекс

5

Проверить

снять план ещё раз

MAGNIT TECH

Моделирование аналитических данных

Факты и измерения

Строка `order_items`: «заказ 117, товар Cable A, **2 шт., цена 100, скидка 100**». Числа — это факт. Какой товар, что заказ сделал клиент 9 из Москвы 10 марта 2026 — это измерения.

	Факты	Измерения
Что хранят	события и числа: позиции заказов, платежи, клики	описания: товары, клиенты, даты
Форма таблицы	узкие и очень длинные	широкие и короткие
На какой вопрос	«сколько?»	«кто, что, где, когда?»
Пример из shop	`order_items`, `payments`, `events`	`customers`, `products`, `product_categories`
Как назовём в хранилище	`fact_order_items`, `fact_payments`	`dim_customer`, `dim_product`, `dim_date`

Зерно факта, или grain

первое решение при проектировании — что означает **одна строка** таблицы фактов:

в `fact_order_items` одна строка — один товар в одном заказе;

в `fact_payments` одна строка — одна платёжная операция;

в `fact_orders` одна строка — один заказ за весь его жизненный цикл;

в `fact_events` одна строка — одно действие на сайте.

> *Зерно фиксируем до того, как выбирать столбцы. Самая частая ошибка — смешать разное зерно в одной таблице.*

Смешанное зерно: 5200 вместо 3100

order_id	total_amount	payment_id	status	amount
113	200.00	1015	success	200.00
113	200.00	1016	success	200.00
116	1900.00	1019	success	1000.00
116	1900.00	1020	success	900.00
119	1000.00			

У заказа 113 две успешные оплаты по 200, у 116 — оплата частями: 1000 и 900.

Сумма заказа (зерно «заказ») оказалась рядом с платежами (зерно «операция»).

✅ *В витрине практики сначала свели платежи до зерна заказа и только потом соединили.*

Виды таблиц фактов

Событие

Одна строка на событие. Самый частый вид.

Пример:
в shop: order_items, payments, events

Снимок на дату

Состояние на конец дня или месяца: остаток на складе, баланс счёта.

Пример:
в shop такой таблицы нет

Процесс

Одна строка на процесс, дополняется по этапам: создан → оплачен → отгружен → доставлен.

Пример:
заказ 113 создан 8 марта в 20:00, оплачен в 20:03

Ключи измерений

Натуральный

Идентификатор из системы-источника:

`customer_id = 1` из базы shop

Суррогатный

Собственный числовой ключ хранилища:

`customer_sk = 41`

dim_date — одна строка на день, в столбцах уже посчитаны год, квартал, неделя, день недели, выходной, праздник. Праздники формулой не посчитаешь.

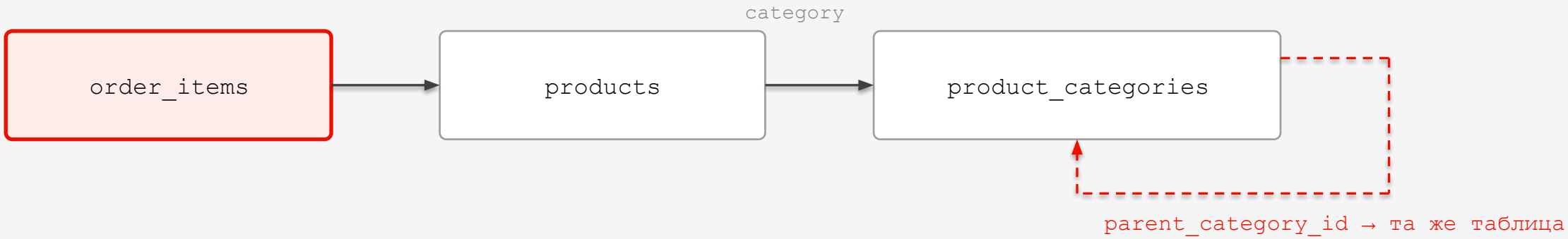
Таблицы хранилища складываем в отдельную схему `dwh`.

```sql
create schema dwh;

create table dwh.dim_date as
select d::date                          as date_sk,
       extract(year    from d)::int     as year,
       extract(quarter from d)::int     as quarter,
       extract(month   from d)::int     as month,
       extract(isodow  from d)::int     as day_of_week,
       extract(isodow  from d) in (6, 7)
                                        as is_weekend
from generate_series(date '2026-01-01',
       date '2026-12-31', interval '1 day') as d;
```

Наша база shop — снежинка

В снежинке описание разбито на цепочку таблиц: товар → категория → родительская категория.



Smartphones → Electronics · 2 уровня

Cookware → Kitchen → Home · 3 уровня

Глубина разная — посмотрим, во что это превращается в запросе.

Запрос к снежинке: шесть таблиц

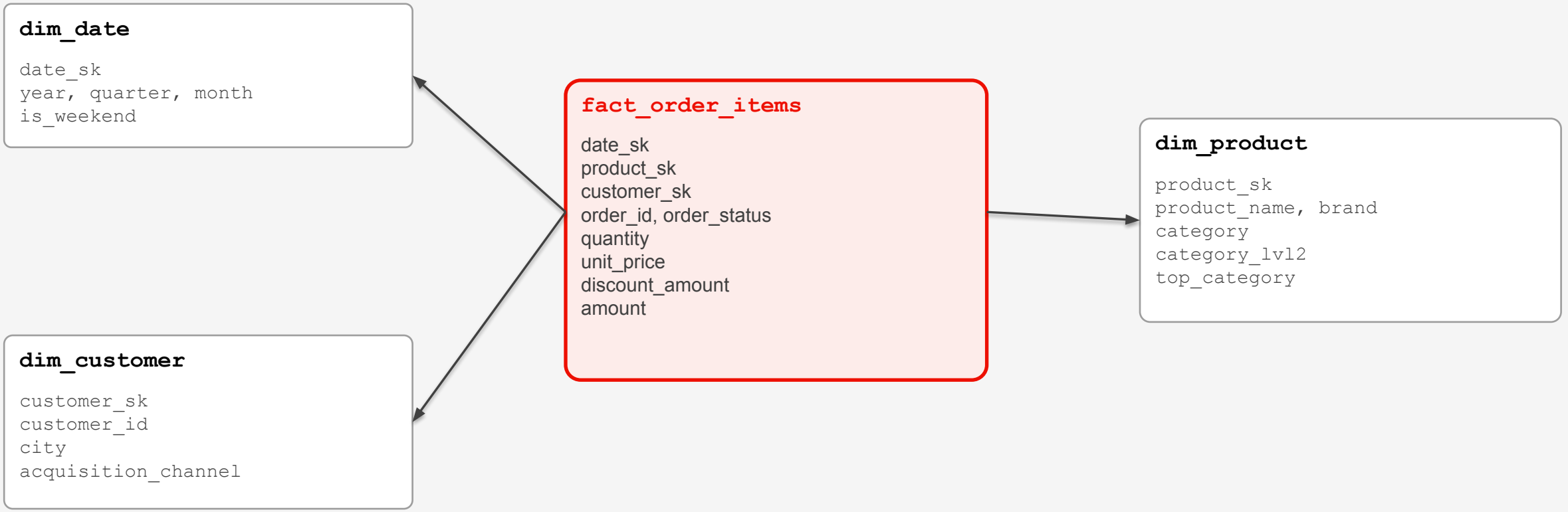
«Какая выручка по верхним категориям в каждом городе по месяцам?» Для ответа — шесть таблиц, `product_categories` дважды сама с собой.

```sql
select date_trunc('month', o.order_created_at)::date as month,
       coalesce(top.category_name, parent.category_name,
                cat.category_name) as top_category,
       c.city,
       sum(oi.quantity * oi.unit_price - oi.discount_amount) as revenue
from order_items oi
join orders o                on o.order_id = oi.order_id
join customers c             on c.customer_id = o.customer_id
join products p              on p.product_id = oi.product_id
join product_categories cat on cat.category_name = p.category
left join product_categories parent
       on parent.category_id = cat.parent_category_id
left join product_categories top
       on top.category_id = parent.parent_category_id
where o.status = 'paid'
group by 1, 2, 3;
```

month	top	city	revenue
2026-03	Electronics	Kazan	6780.35
2026-03	Electronics	Moscow	6222.30
2026-03	Home	Kazan	2199.65

Запрос работает, но писать его каждый раз неудобно и легко ошибиться. Дальше — как это упростить.

Схема «звезда» для shop



В звезде каждое измерение — одна таблица со всем описанием. Соберём её из таблиц shop в схеме dwh.

Строим dim_product: расплющиваем дерево

Звезду собираем из таблиц shop в отдельной схеме `dwh` — чтобы не смешивать с исходными данными и витринами практики.

Главная работа — расплющить дерево категорий. Глубина разная, поэтому второй уровень зависит от того, есть ли у товара третий.

Сложное соединение пишем **один раз** — при загрузке. Аналитикам его больше не повторять.

```sql
create table dwh.dim_product as
select p.product_id      as product_sk,
       p.product_name,
       p.brand,
       cat.category_name as category,
       case when top.category_name is not null
            then parent.category_name  -- 3 уровня
            else cat.category_name     -- 2 уровня
       end as category_lvl2,
       coalesce(top.category_name, parent.category_name,
                cat.category_name) as top_category
from products p
join product_categories cat
  on cat.category_name = p.category
left join product_categories parent
  on parent.category_id = cat.parent_category_id
left join product_categories top
  on top.category_id = parent.parent_category_id;
```

13

товаров в dim_product

2 или 3

уровня в дереве категорий

Smartphones → Electronics: 2 уровня. Cookware → Kitchen → Home: 3 уровня.

MAGNIT TECH

Факт и измерение клиентов

Факт: зерно — позиция заказа

```sql
create table dwh.fact_order_items  as
select o.order_created_at::date  as date_sk,
       oi.product_id   as product_sk,
       o.customer_id   as customer_sk,
       oi.order_id,
       o.status  as order_status,
       oi.quantity, oi.unit_price,
       oi.discount_amount,
       oi.quantity * oi.unit_price
         - oi.discount_amount  as amount
from order_items oi
join orders o  on o.order_id = oi.order_id;
```

Клиенты: пока без истории

```sql
create table dwh.dim_customer  as
select customer_id  as customer_sk,
       customer_id,
       city,
       acquisition_channel
from customers;

-- проверка зерна
select count(*)  from dwh.fact_order_items;  -- 444
select count(*)  from order_items;           -- 444
```

`444 = 444`: соединение с orders ничего не размножило и не потеряло — зерно правильное. `order_id` и `order_status` оставили в факте, чтобы по ним было удобно фильтровать.

MAGNIT TECH∞

Запрос к звезде: те же цифры

product_sk	product_name	brand	category	category_lvl2	top_category
1	Phone A	North	Smartphones	Smartphones	Electronics
11	Pot A	East	Cookware	Kitchen	Home
12	Case B	West	Cases	Accessories	Electronics

```sql
select d.year, d.month, p.top_category, c.city,
       sum(f.amount)  as revenue
from dwh.fact_order_items f
join dwh.dim_date d      on d.date_sk     = f.date_sk
join dwh.dim_product p   on p.product_sk  = f.product_sk
join dwh.dim_customer c  on c.customer_sk = f.customer_sk
where f.order_status = 'paid'
group by 1, 2, 3, 4;
```

Сверка со снежинкой через `except`: по **46 строк**, расхождений **0**.

Без двойного соединения категорий и coalesce. Запросы к звезде однотипны — BI строит их сам.

Звезда или снежинка

	Звезда	Снежинка
JOIN в запросе	меньше, один уровень	больше, цепочки (в shop — до трёх уровней категорий)
Дублирование	есть: «Electronics» повторяется у каждого товара	почти нет: каждая категория — одна строка
Переименовать категорию	нужно обновить много строк	одна строка
Понятность для аналитика	выше	ниже



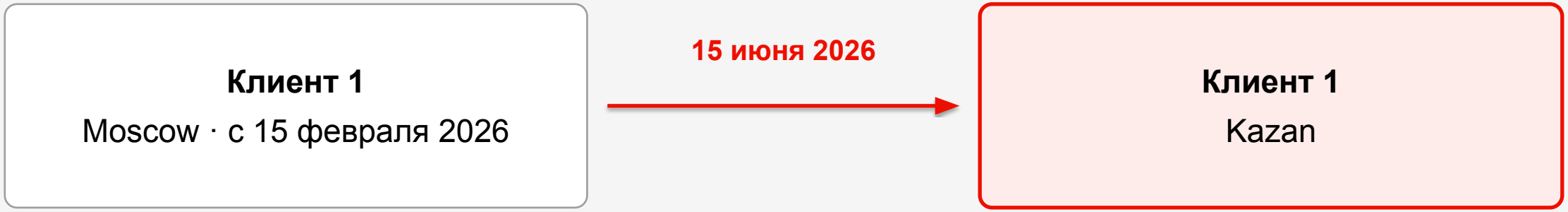
*По умолчанию выбирают звезду: измерения намного меньше фактов — у нас **13 товаров против 444 позиций заказов**. Снежинка — когда измерения огромные или одно дерево категорий нужно нескольким измерениям.*

MAGNIT TECH

Что делать, если данные в измерении меняются?

Атрибуты измерений меняются

Клиент переезжает, товар переводят в другую категорию. Меняются редко — slowly changing dimensions, SCD.



*«Какая была выручка по городам за первое полугодие?» Куда отнести его оплаченные заказы с марта по май — **8 заказов на 6 730**?*

SCD Type 1: перезапись

Старое значение затирается новым.

	customer_sk	customer_id	city	acquisition_channel
было	1	1	Moscow	search
стало	1	1	Kazan	search

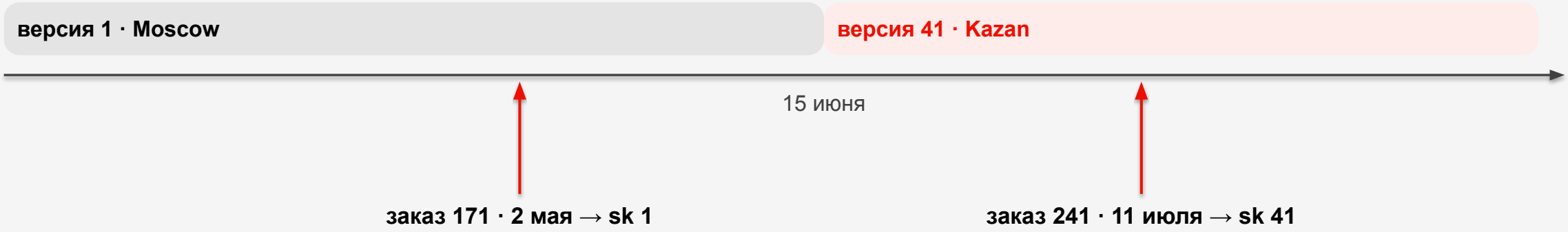
✅ **Плюс:** просто, таблица не растёт. Подходит для опечаток и атрибутов без ценной истории.

🔖 **Минус:** все покупки «переедут» в Казань — 6 730 уйдут из Москвы. Отчёт за полугодие в июне и в июле покажет разные цифры.

SCD Type 2: новая версия строки

Старую строку закрываем, добавляем новую версию с новым суррогатным ключом.

customer_sk	customer_id	city	channel	valid_from	valid_to	is_current
1	1	Moscow	search	2026-02-15 10:00	2026-06-15	false
41	1	Kazan	search	2026-06-15	9999-12-31	true



`9999-12-31` *значит «действует до сих пор». Прошлые цифры отчёта не изменятся.*

MAGNIT TECH

Заказ находит свою версию

```sql
select o.order_id, o.order_created_at::date,
       d.customer_sk, d.city
from orders o
join dim_customer d
  on d.customer_id = o.customer_id
 and o.order_created_at >= d.valid_from
 and o.order_created_at <  d.valid_to
where o.order_id in (171, 241);
```

```sql
create table dim_customer (
  customer_sk bigint  generated always
              as identity primary key,
  customer_id         int   not null,
  city                text  not null,
  acquisition_channel text  not null,
  valid_from timestamp  not null,
  valid_to   timestamp  not null
             default '9999-12-31',
  is_current boolean  not null default true
);
```



Интервалы версий не должны пересекаться и иметь дыр — иначе заказ найдёт две версии или ни одной. Поэтому `valid_from` включаем (>=), а `valid_to` — нет (<).

Загрузка Type 2 в два шага

Шаг 1. Закрыть изменившиеся версии

```
update dim_customer d
set valid_to = :load_ts,
    is_current = false
from stg_customers s
where d.customer_id = s.customer_id
  and d.is_current
  and (d.city, d.acquisition_channel)
      is distinct from
      (s.city, s.acquisition_channel);
```

Шаг 2. Вставить новые версии

```
insert into dim_customer
  (customer_id, city,
   acquisition_channel, valid_from)
select s.customer_id, s.city,
       s.acquisition_channel, :load_ts
from stg_customers s
where not exists (
  select 1 from dim_customer d
  where d.customer_id = s.customer_id
    and d.is_current);
```

`is distinct from` — это «не равно», которое правильно работает с пустыми значениями (`null`). Обычное `<>` с null не срабатывает, и изменение потерялось бы. `stg_customers` — свежая выгрузка клиентов из shop.

На практике типы смешивают

- опечатку в названии города исправляют **перезаписью** (Type 1), а переезд **версионируют** (Type 2);
- версии создают только по важным атрибутам, иначе таблица растёт от каждой мелочи;
- в dbt для Type 2 есть готовый механизм — **snapshots**;
- другие типы SCD (0, 3, 6) встречаются реже.

Итоги

- Запрос проходит Parser, Analyze, Rewriter, Planner и Executor. По ошибке видно, где он остановился.
- Плохая статистика — плохой план: 253 строки вместо 333 700, и сортировка уходит на диск.
- EXPLAIN читаем снизу вверх, в параллельных планах умножаем на loops, сравниваем прогноз строк с фактом.
- Индекс помогает, когда строк мало: 108 из 5 млн — 163 мс превращаются в 0,1 мс. Для 76 % строк и таблицы в 2 страницы он не нужен.
- Факты — события с числами, измерения — контекст. Сначала зерно.
- Звезда — по умолчанию; shop — снежинка, и из неё строится звезда с теми же цифрами. Type 1 теряет историю, Type 2 хранит её версиями.

MAGNIT TECH

Спасибо за занятие!